app.py

```
"""CAIAS Student Registration System
Built with Streamlit (pure Python) + SQLite.
Made by Bharath K
"""

import os
import re
import sqlite3
from datetime import datetime

import pandas as pd
import streamlit as st

# ---------------------------------------------------------------- settings
COLLEGE = "CAIAS"
PROGRAMS = ["BCA Full Stack", "BCA Data Science", "BCA AI & ML"]
DB_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "students.db")

REG_PATTERN = re.compile(r"^CAUG\d{2}-\d{4}$")          # e.g. CAUG26-0240
PHONE_PATTERN = re.compile(r"^[6-9]\d{9}$")              # 10-digit Indian mobile
EMAIL_PATTERN = re.compile(r"^[a-z0-9._%+\-]+@gmail\.com$")

RATINGS = ["\U0001f61e Not really", "\U0001f610 It's okay", "\U0001f642 Good", "\U0001f600 Great", "\
  U0001f929 Love it!"]
LIKE_MOST = ["Faculty", "Campus", "Labs & Lab facilities", "Friends", "Events & Fests",
             "Placements", "Something else"]
CLUBS = ["Coding Club", "Tech Fest Team", "Sports", "Cultural / Dance / Music",
         "Photography", "None right now"]


# ---------------------------------------------------------------- validation
def validate(name, reg_no, phone, email, program):
    """Return (clean_values, list_of_errors)."""
    errors = []
    name = " ".join(name.split())
    reg_no = reg_no.strip().upper()
    phone = phone.strip().replace(" ", "")
    email = email.strip().lower()

    if len(name) < 3 or not re.fullmatch(r"[A-Za-z .'\-]+", name):
        errors.append("Enter your full name using letters only (at least 3 characters).")
    if not REG_PATTERN.match(reg_no):
        errors.append("Register number must look like CAUG26-0240 "
                      "(CAUG + 2-digit year + dash + 4 digits).")
    if not PHONE_PATTERN.match(phone):
        errors.append("Phone number must be a 10-digit mobile number starting with 6, 7, 8 or 9.")
    if not EMAIL_PATTERN.match(email):
        errors.append("Enter a valid Gmail address ending with @gmail.com.")
    if program not in PROGRAMS:
        errors.append("Please choose your BCA program.")
    return (name, reg_no, phone, email), errors


# ---------------------------------------------------------------- database
def get_conn():
    return sqlite3.connect(DB_PATH)


def init_db():
    with get_conn() as conn:
        conn.execute(
            """
            CREATE TABLE IF NOT EXISTS registrations (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                reg_no TEXT NOT NULL UNIQUE,
                phone TEXT NOT NULL,
                email TEXT NOT NULL,
                program TEXT NOT NULL,
                rating TEXT,
                like_most TEXT,
                clubs TEXT,
                suggestion TEXT,
                registered_at TEXT NOT NULL
            )
```

```
            """
        )


def save_registration(values):
    with get_conn() as conn:
        conn.execute(
            """
            INSERT INTO registrations
            (name, reg_no, phone, email, program, rating, like_most, clubs, suggestion, registered_at)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """,
            values,
        )


def load_registrations():
    with get_conn() as conn:
        return pd.read_sql_query(
            "SELECT id, name, reg_no, phone, email, program, rating, like_most, clubs, "
            "suggestion, registered_at FROM registrations ORDER BY id DESC",
            conn,
        )


# ---------------------------------------------------------------- admin password
def admin_password():
    try:
        if "ADMIN_PASSWORD" in st.secrets:
            return st.secrets["ADMIN_PASSWORD"]
    except Exception:
        pass
    return os.environ.get("ADMIN_PASSWORD", "admin123")  # change this!


# ---------------------------------------------------------------- styling
def inject_css():
    st.markdown(
        """
        <style>
        .block-container {max-width: 560px; padding-top: 1.2rem;}
        .hero {
            background: linear-gradient(135deg, #5B3FD9 0%, #8E54E9 55%, #FF6FB5 100%);
            color: white; padding: 26px 20px; border-radius: 20px;
            text-align: center; margin-bottom: 18px;
            box-shadow: 0 8px 24px rgba(91,63,217,.3);
        }
        .hero h1 {margin: 0; font-size: 1.7rem; color: white;}
        .hero p {margin: 6px 0 0; opacity: .92;}
        .thanks {
            background: linear-gradient(135deg, #e9e3ff, #ffe3f1);
            border-radius: 20px; padding: 28px 18px; text-align: center;
            color: #2b1f5c;
        }
        .thanks h2 {color: #5B3FD9; margin-top: 0;}
        .madeby {text-align: center; color: #7a7a8c; margin-top: 22px; font-size: .9rem;}
        div[data-testid="stFormSubmitButton"] button {
            background: linear-gradient(135deg, #5B3FD9, #8E54E9);
            color: white; border: 0; border-radius: 12px; font-weight: 600;
        }
        </style>
        """,
        unsafe_allow_html=True,
    )


def hero(subtitle):
    st.markdown(
        f'<div class="hero"><h1>\U0001f393 {COLLEGE} Student Registration</h1><p>{subtitle}</p></div>',
        unsafe_allow_html=True,
    )


def footer():
    st.markdown('<div class="madeby">Made by Bharath K</div>', unsafe_allow_html=True)
# ---------------------------------------------------------------- pages
def registration_page():
```

```
    if st.session_state.get("done"):
        st.balloons()
        st.markdown(
            f"""
            <div class="thanks">
              <h2>\U0001f389 Thank you, {st.session_state['done_name']}!</h2>
              <p>Your registration for <b>{st.session_state['done_program']}</b> is complete.</p>
              <p>We're glad to have you at {COLLEGE}. Keep learning, keep building! \U0001f680</p>
              <p><b>Made by Bharath K</b></p>
            </div>
            """,
            unsafe_allow_html=True,
        )
        if st.button("Register another student"):
            for k in ("done", "done_name", "done_program"):
                st.session_state.pop(k, None)
            st.rerun()
        return

    hero("Fill in your details to register")

    with st.form("registration_form"):
        st.subheader("\U0001f4dd Your details")
        name = st.text_input("Full name", placeholder="e.g. Ashwin Kumar")
        reg_no = st.text_input("College register number", placeholder="CAUG26-0240",
                               max_chars=10, help="Format: CAUG + year + dash + 4 digits")
        phone = st.text_input("Phone number", placeholder="10-digit mobile number", max_chars=10)
        email = st.text_input("Gmail ID", placeholder="yourname@gmail.com")
        program = st.selectbox("Which BCA program are you studying?", ["Select..."] + PROGRAMS)

        st.subheader("\U0001f4ac A few quick questions")
        rating = st.select_slider("Do you like the college?", options=RATINGS, value=RATINGS[2])
        like_most = st.selectbox("What do you like most about the college?", LIKE_MOST)
        clubs = st.multiselect("Which clubs or activities would you join?", CLUBS)
        suggestion = st.text_area("Any suggestions for the college? (optional)", max_chars=300)

        submitted = st.form_submit_button("Register \U00002705", use_container_width=True)

    if submitted:
        (name, reg_no, phone, email), errors = validate(name, reg_no, phone, email, program)
        if errors:
            for e in errors:
                st.error(e)
        else:
            try:
                save_registration((
                    name, reg_no, phone, email, program, rating, like_most,
                    ", ".join(clubs), suggestion.strip(),
                    datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                ))
            except sqlite3.IntegrityError:
                st.error(f"The register number {reg_no} is already registered.")
            else:
                st.session_state["done"] = True
                st.session_state["done_name"] = name.split()[0]
                st.session_state["done_program"] = program
                st.rerun()
    footer()


def admin_page():
    hero("Admin dashboard")
    pw = st.text_input("Admin password", type="password")
    if not pw:
        return
    if pw != admin_password():
        st.error("Wrong password.")
        return

    df = load_registrations()
    st.success(f"Total registrations: {len(df)}")
    if df.empty:
        return

    st.subheader("By program")
    st.bar_chart(df["program"].value_counts())
    st.subheader("All registrations")
```

```
    st.dataframe(df, use_container_width=True, hide_index=True)
    st.download_button("\U00002b07\U0000fe0f Download CSV", df.to_csv(index=False).encode("utf-8"),
                       file_name="registrations.csv", mime="text/csv",
                       use_container_width=True)


# ---------------------------------------------------------------- main
def main():
    st.set_page_config(page_title=f"{COLLEGE} Registration", page_icon="\U0001f393")
    init_db()
    inject_css()

    page = st.sidebar.radio("Menu", ["Register", "Admin"])
    if page == "Register":
        registration_page()
    else:
        admin_page()


if __name__ == "__main__":
    main()
```

requirements.txt

```
streamlit
pandas
```